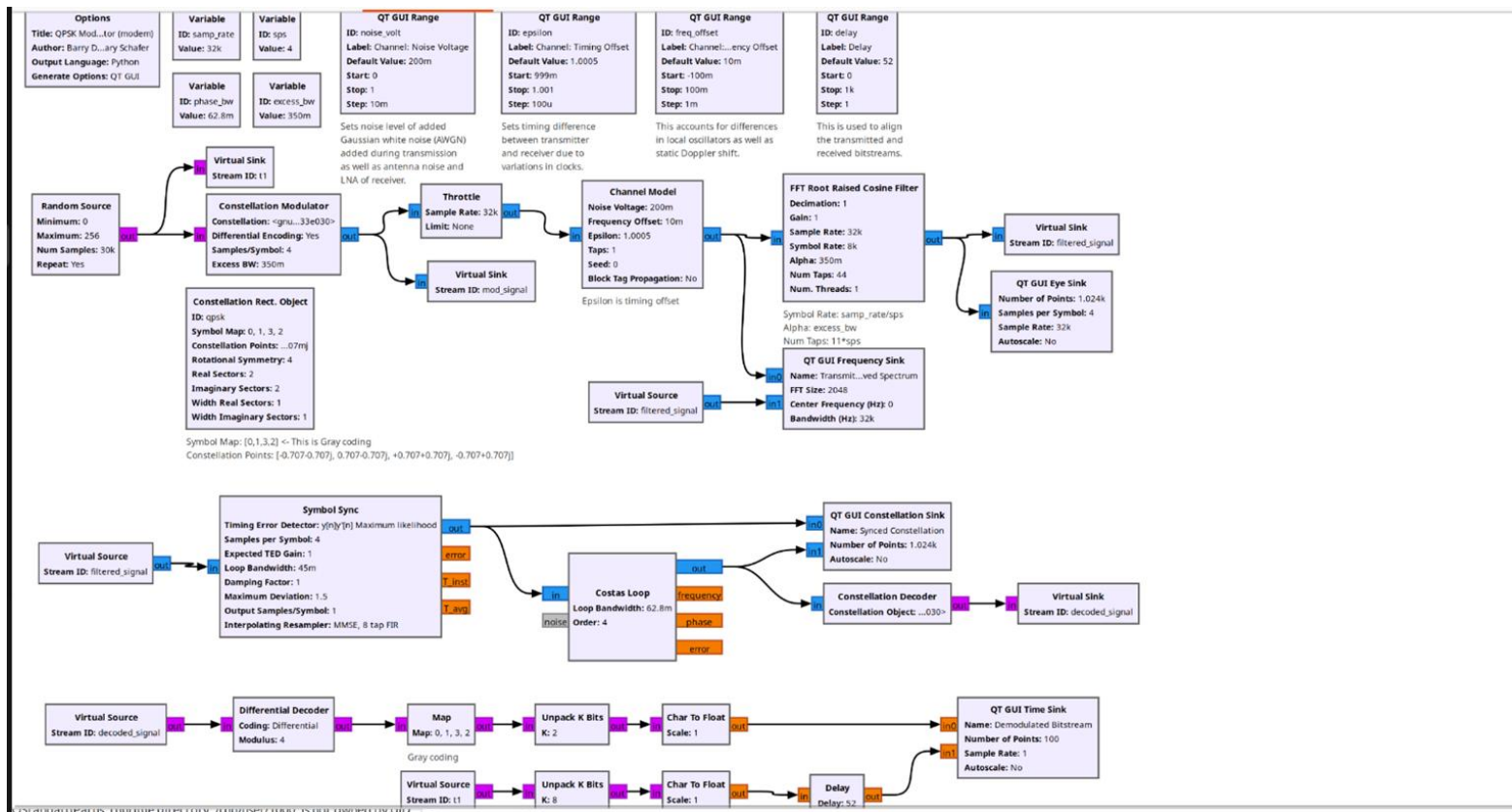


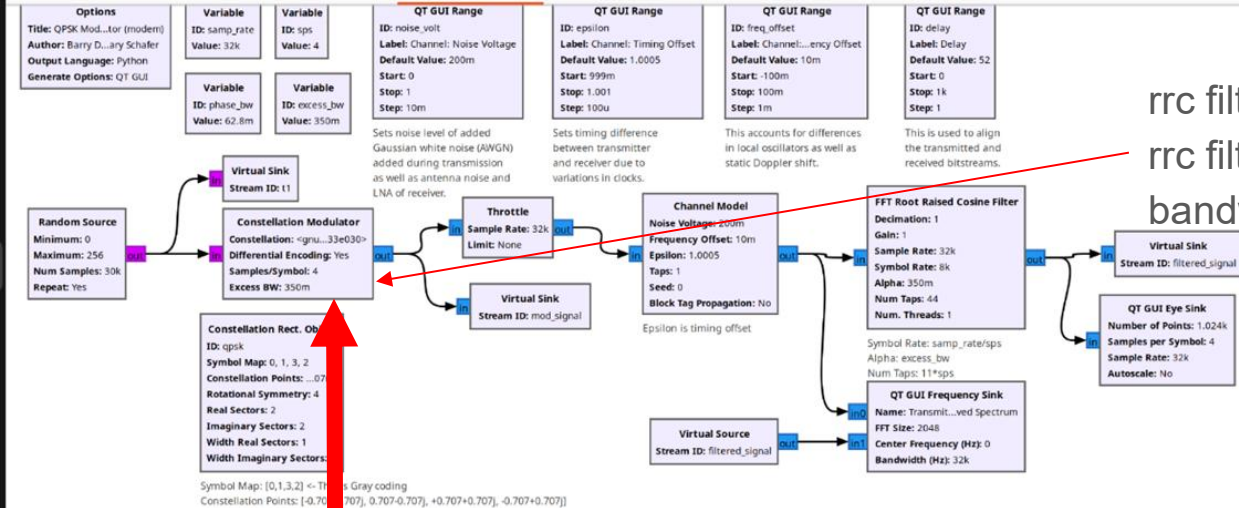
QPSK

26.1.29
정민석

gnuradio qpsk tutorial sim



tx단



rrc filtering 기능이 있어,
rrc filtered 출력이 나옴. 그래서 excess
bandwidth 파라미터 존재

Properties: Constellation Modulator

General Advanced Documentation

Constellation qpsk

Differential Encoding Yes [bool]

Samples/Symbol sps [int]

Excess BW excess_bw [real]

Verbose Off [bool]

Log Off [bool]

Truncate Filter Transient No [bool]

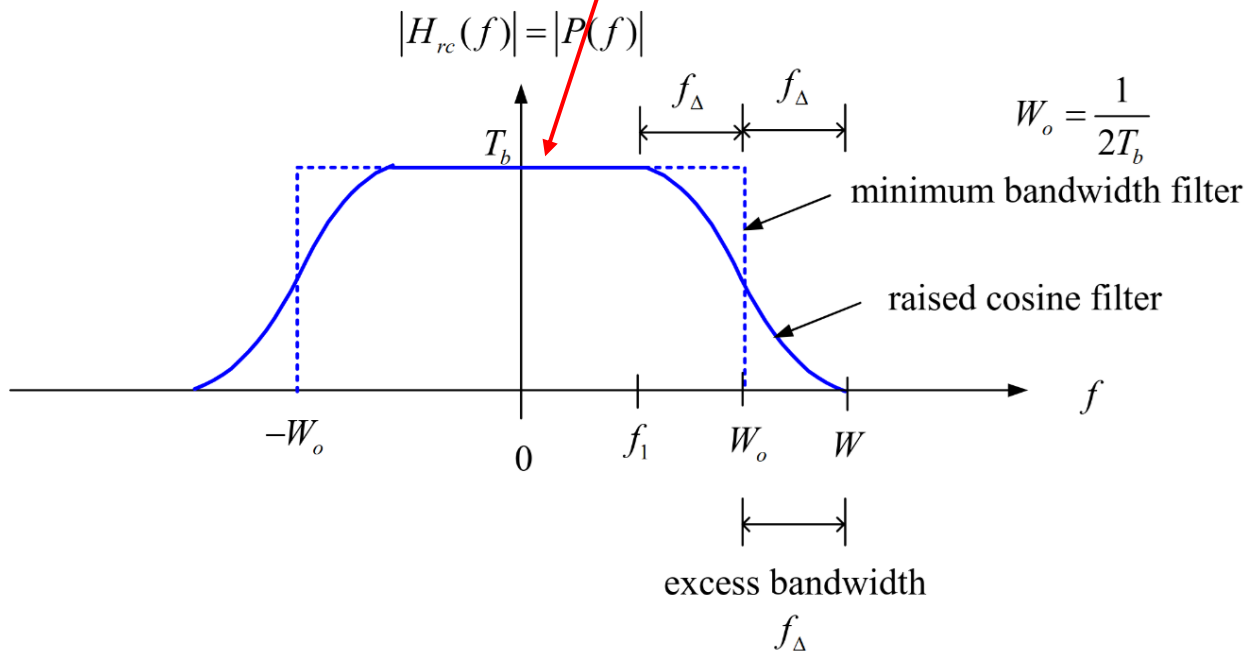
확인(O) 취소(C) 적용(A)

qpsk 변조 블록 사용
, 차동 인코딩 방식 사용
, sample per symbol은 4이므로
 $1\text{sym}=2\text{bits}$, $1\text{bits}=2\text{sample}$
, excess bw은 0.35
, truncate filter transient(과도구간 자르기)

excess bw

$$P(f) = \text{transmit filter}(f) * \text{channel}(f) * \text{receive filter}(f) \\ = H_{\text{channel}}(f) * H_{\text{rc}}(f)$$

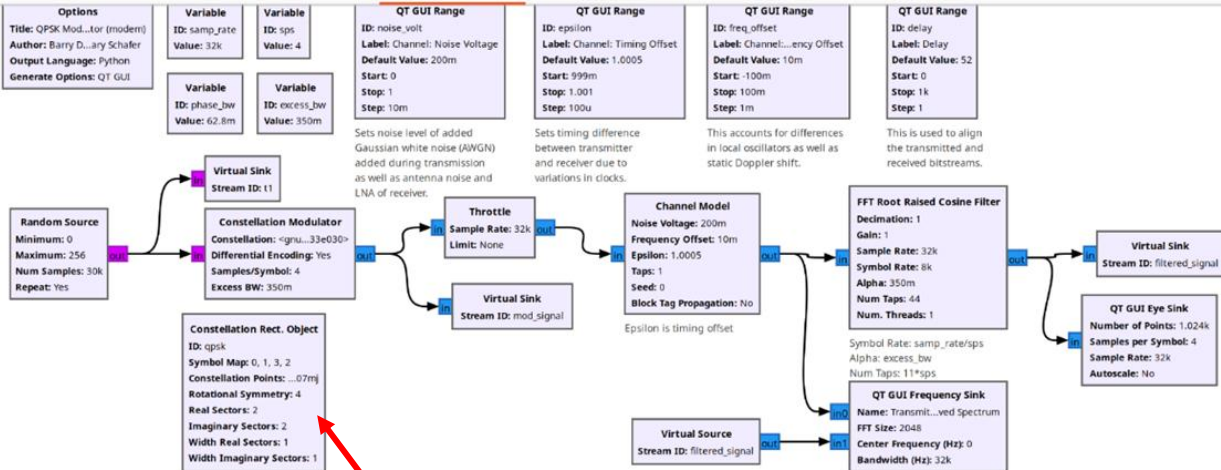
- 이론적 최소 대역폭 W_o 를 넘는 추가 대역폭 f_{Δ} 를 초과 대역폭(**excess bandwidth**)이라 한다.



excess bandwidth와 roll of factor

excess BW는 최소 나이퀴스트 대역에서 얼마나 더 대역폭을 쓰는지에 대한 값으로 excess BW가 35%는 roll of factor alpha가 35%라는 것과 동일.

symbol rate R_s 일 때 나이퀴스트 최소 대역폭은 $R_s/2$ 이고 excess BW를 나이퀴스트 대역폭의 α 배라고 하면 총 대역폭 = $R_s/2 + R_s/2 * \alpha = R_s/2 * (1 + \alpha)$



Symbol Map: [0,1,3,2] <- This is Gray coding

Properties: Constellation Rect. Object

General

Advanced

Documentation

Symbol Map

[0, 1, 3, 2]

[int_vector]

Constellation Points

[-0.707-0.707j, 0.707-0.707j, +0.707+0.707j, -0.707+0.707j]

[complex_vector]

Rotational Symmetry

4

[int]

Real Sectors

2

[int]

Imaginary Sectors

2

[int]

Width Real Sectors

1

[int]

Width Imaginary Sectors

1

[int]

Soft bits precision

8

[int]

Soft Decisions LUT

None

확인(O)

취소(C)

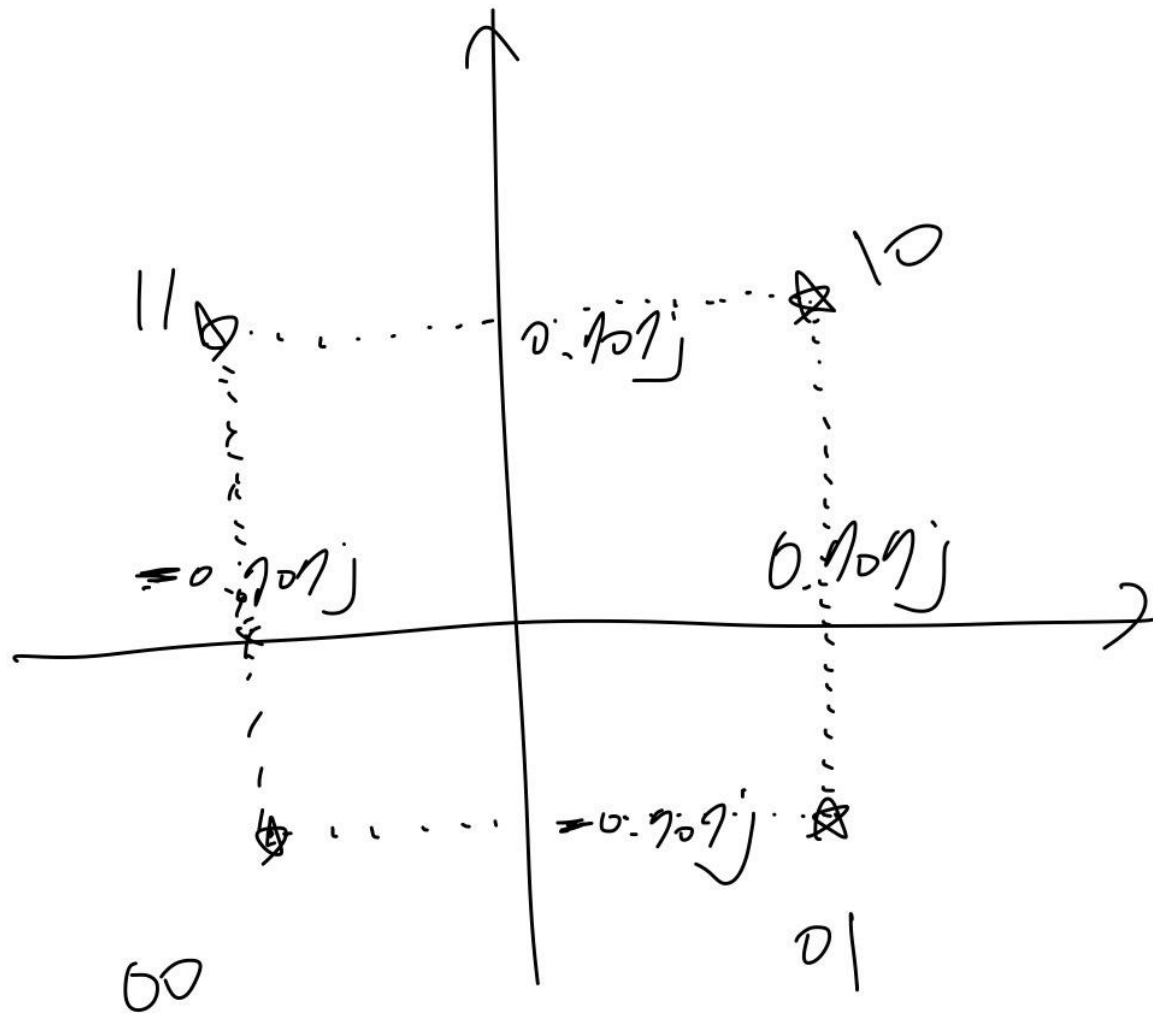
적용(A)

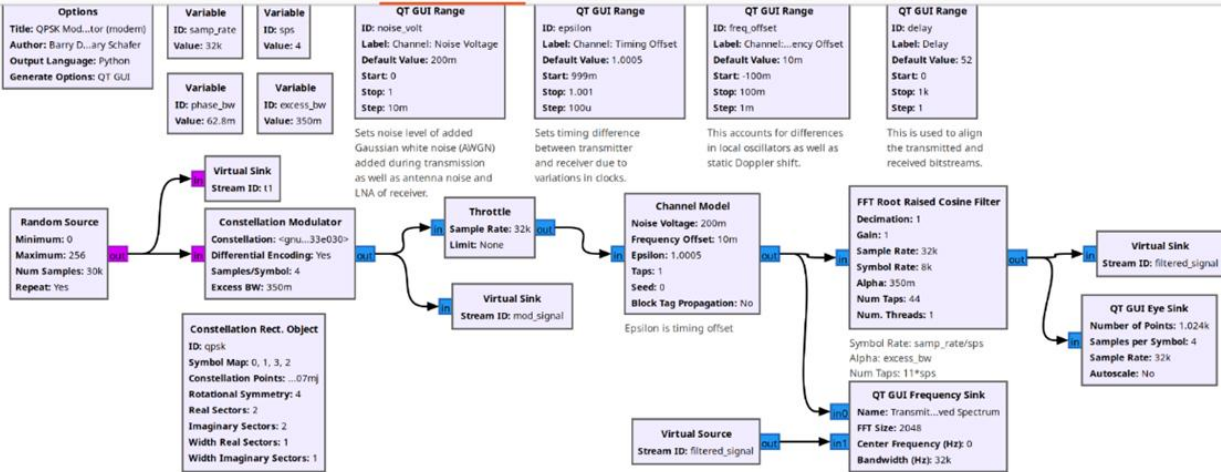
<심볼맵→성상도 지점>

0	→	-0.707-0.707j
1	→	0.707-0.707j
3	→	-0.707+0.707j
2	→	0.707+0.707j

<비트→심볼 맵>

00	→	0
01	→	1
11	→	3
10	→	2





Symbol Map: [0,1,3,2] <- This is Gray coding

Properties: Constellation Rect. Object

General	Advanced	Documentation
Symbol Map	[0, 1, 3, 2]	[int_vector]
Constellation Points	[-0.707-0.707j, 0.707-0.707j, +0.707+0.707j, -0.707+0.707j]	[complex_vector]
Rotational Symmetry	4	[int]
Real Sectors	2	[int]
Imaginary Sectors	2	[int]
Width Real Sectors	1	[int]
Width Imaginary Sectors	1	[int]
Soft bits precision	8	[int]
Soft Decisions LUT	None	

확인(O) 취소(C) 적용(A)

회전했을 때 같은 모양이 나오는 경우의 수 4가지.

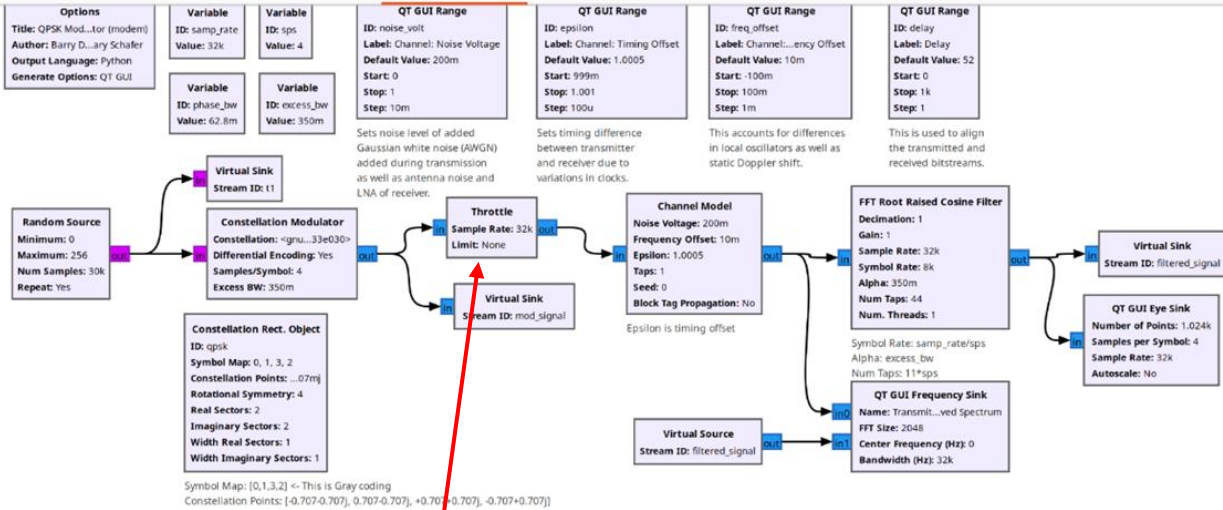
, 실수 영역을 2가지로 분류(0보다 큰 영역, 0보다 작은 영역).

, 허수 영역을 2가지로 분류(0보다 큰 영역, 0보다 작은 영역).

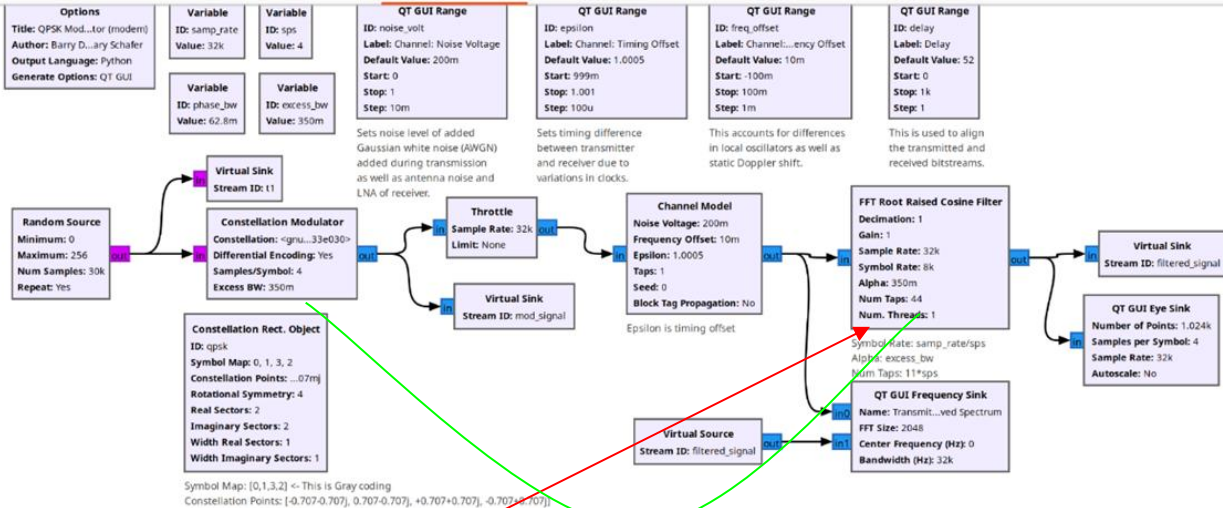
, decision region의 boudary를 설정하기 위한 실수 영역의 폭.

, decision region의 boudary를 설정하기 위한 허수 영역의 폭.

예를 들어서 real sectors=2, imag sector=2, width real sector=2, width imag sector=2,
이라고 하면 실수축을 $[-2,0) + [0,2)$ 로 2구간으로 자르고
허수축도 $[-2,0) + [0,2)$ 로 자른다.



throttle블록은 설정한 속력으로 샘플을 입력받아 출력으로 복사해서, 샘플들을 소비, 공급 속력을 조절하는 목적의 블록이다. 시뮬레이션 사용할 때 필수



Properties: FFT Root Raised Cosine Filter

General Advanced Documentation

Type: Complex->Complex (Complex Taps) v

Decimation: 1 [int]

Gain: 1 [real]

Sample Rate: samp_rate [real]

Symbol Rate: samp_rate/sps [real]

Alpha: excess_bw [real]

Num Taps: 11*sps [int]

Num. Threads: 1 [int]

확인(O) 취소(C) 적용(A)

11*(symbol 1개당 sample)개의 fir 계수를 갖는다.

필터 연산을 몇개의 쓰레드로 할 것인가

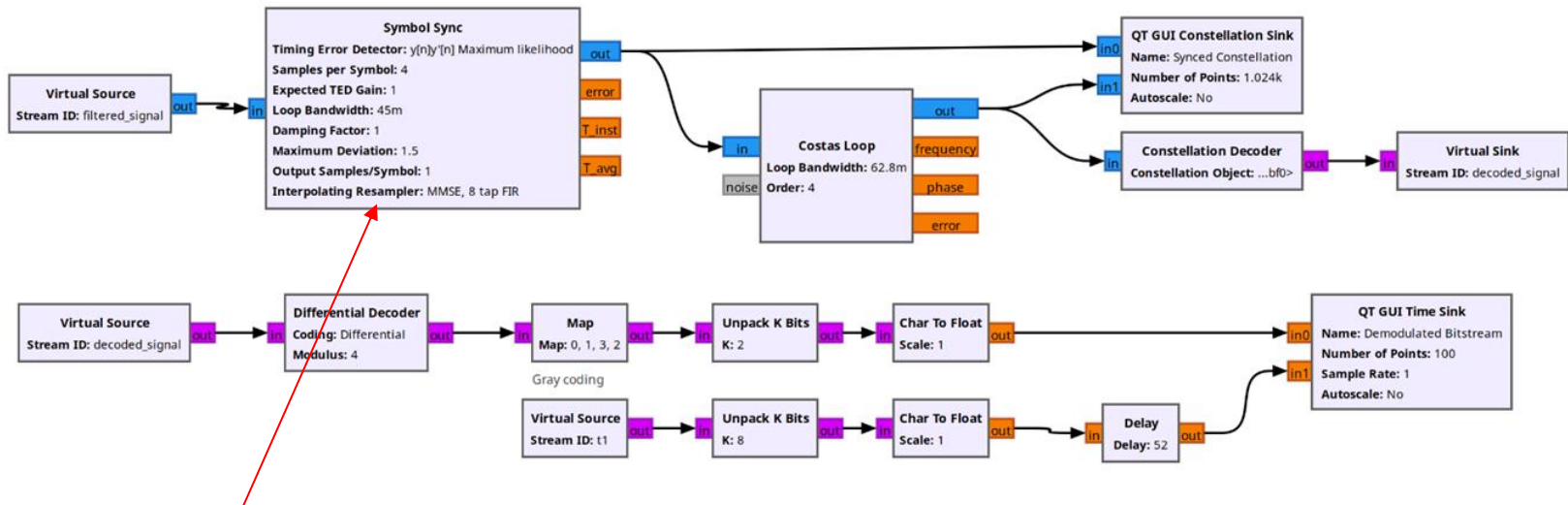
RC & RRC filter

보통 tx에 rrc filter 1개, rx에 rrc filter 1개를 뒀서 rc filter를 만들어서 ISI를 제거한다..

그렇다면 그냥 RC filter 하나 써서 ISI를 제거하지 왜 굳이 번거롭게 2개의 rrc filter로 쪼개어서 사용을 하는가..?

→튜닝의 자유도 확보..?

rx단



clock recovery를 위한 블록.

timing error detector: $y[n]y'[n]$ maximum likelihood 사용

what is clock recovery?

수신되는 직렬 데이터 스트림으로부터,

- 적절한 클럭을 추출하고,
- 그 클럭으로 다시 수신 데이터를 샘플링하여,
- 수신 데이터를 시간적으로 정확히 재구성하는데 사용되는 수신측 회로

what is timing error detector

현재 샘플링 시점의 타이밍 오류를 구해서 오류를 줄이는 방향으로 보정하는 장치

<https://destevez.net/2020/02/simulating-the-ted-gain-for-a-polyphase-matched-filter/>

https://pnsaeta.github.io/Learning_SDR/lesson18.html

<https://wirelesspi.com/what-is-a-symbol-timing-offset-and-how-it-distorts-the-rx-signal/>

yy' maximum likelihood TED implementation at Symbol Sync block

>timing_error_detector.cc

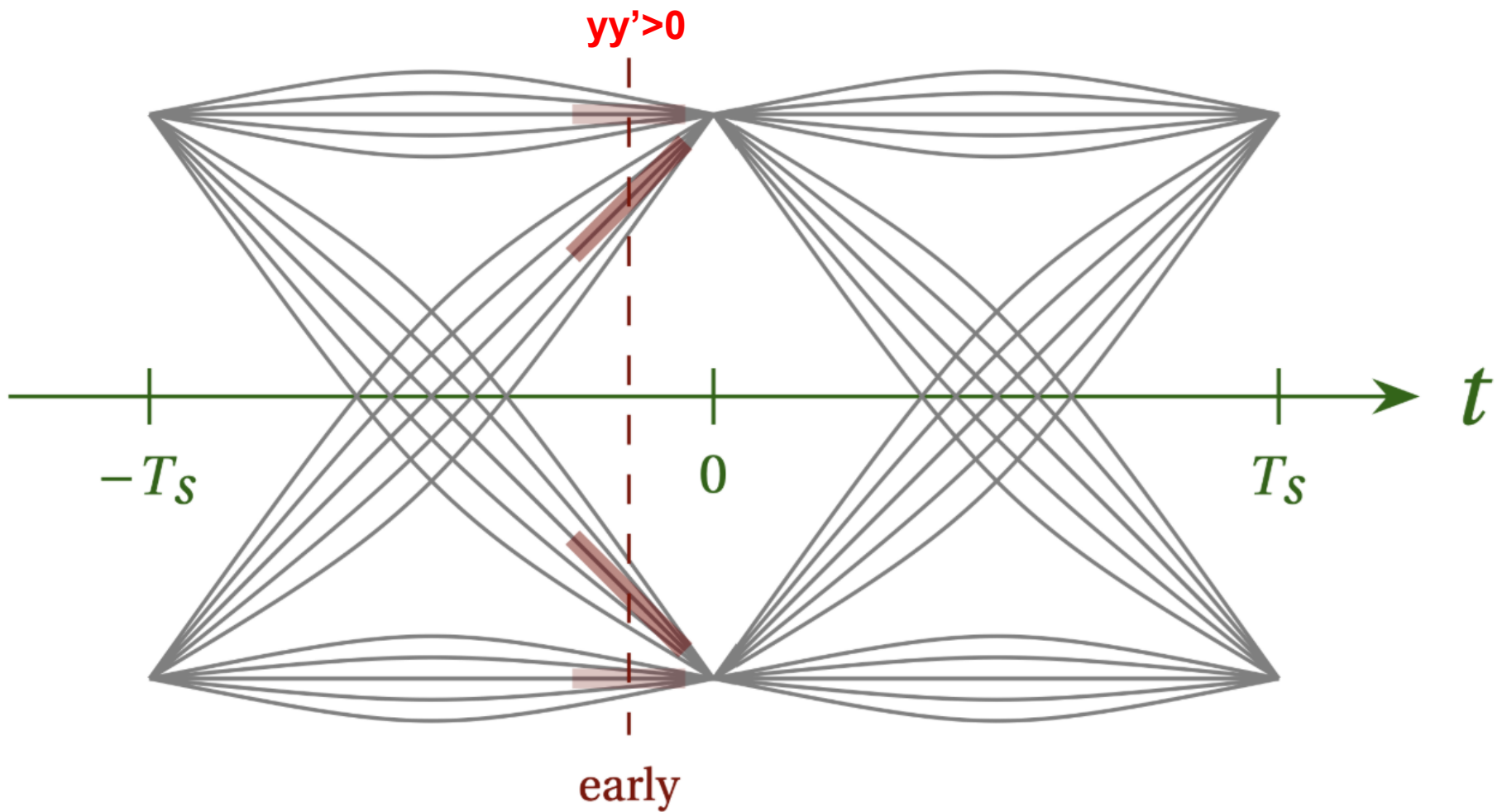
```
float ted_signal_times_slope_ml::compute_error_cf()
```

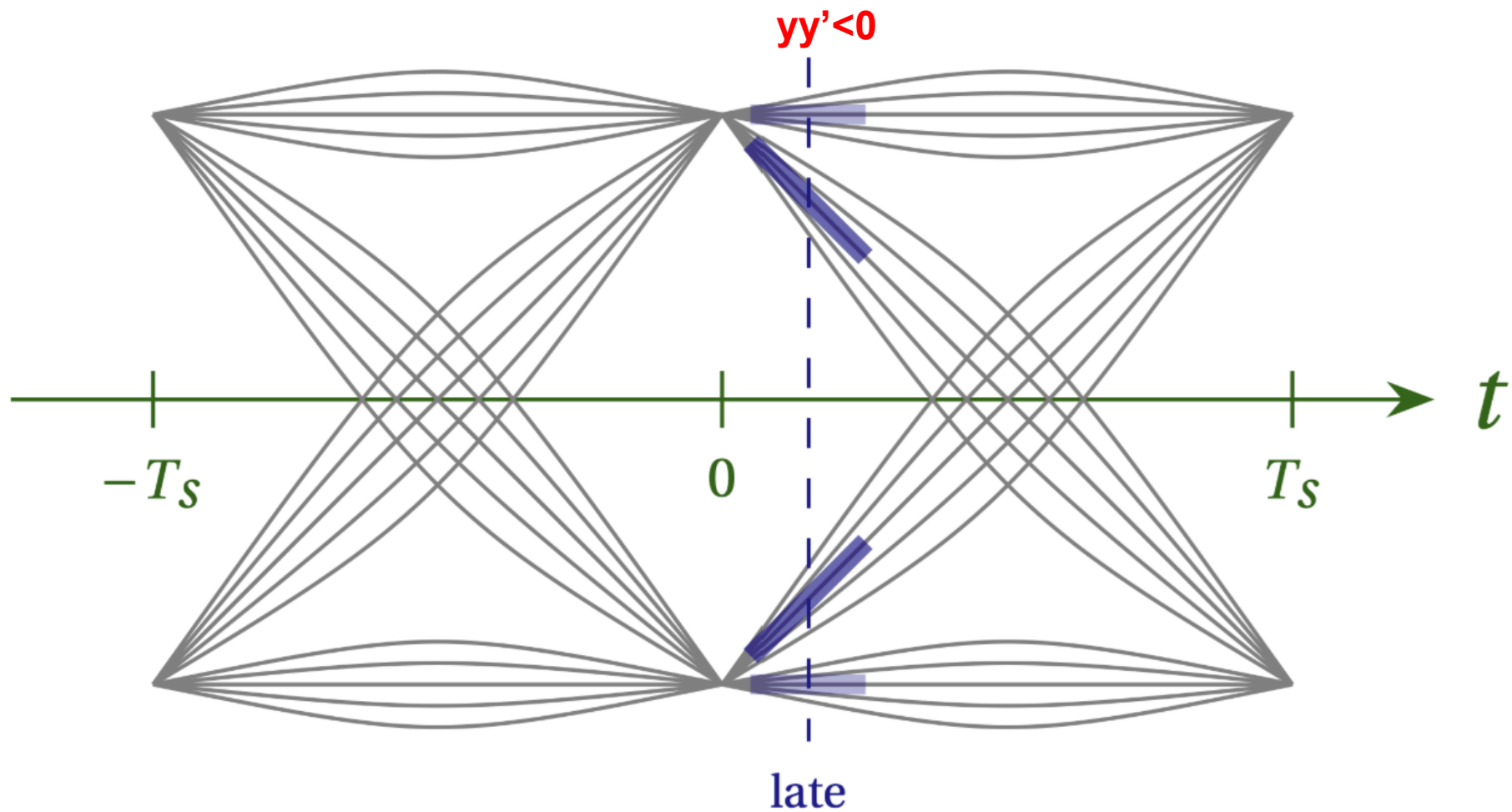
```
{  
    return (d_input[0].real() * d_input_derivative[0].real() +  
            d_input[0].imag() * d_input_derivative[0].imag()) /  
            2.0f;  
}
```

$$e = \langle xx' \rangle$$

```
float ted_signal_times_slope_ml::compute_error_ff()
```

```
{  
    return (d_input[0].real() * d_input_derivative[0].real());  
}
```





Symbol Sync block

>**symbol_sync_cc_impl.cc**

PLL, DLL HW를 SW 코드로 구현.

// 1. 현재 위치 보간

```
interp_output = d_interp->interpolate(&in[ii], d_interp->phase_wrapped());
```

// 2. 미분값 계산

```
if (d_ted->needs_derivative()) // yy' TED는 true 반환
```

```
    interp_derivative = d_interp->differentiate(&in[ii],  
                                                d_interp->phase_wrapped());
```

// 3. TED에 입력 (현재 샘플만)

```
d_ted->input(interp_output, interp_derivative);
```

// 4. 오류 계산

error = d_ted->error(); // 내부적으로 $e = y \times \dot{y}$ 계산


```
if (symbol_clock()) { //심볼 경계 확인
```

```
    d_clock.advance_loop(error); //pi제어기
```

```
        //err>0위상을 앞당기기, err<0위상을 뒤로 미루기, err=0 샘플링 타이밍 정렬!
```

```
    d_inst_clock_period = d_clock.get_inst_period();
```

```
    d_avg_clock_period = d_clock.get_avg_period();
```

```
    d_clock.phase_wrap();
```

```
// Symbol Clock and Interpolator Positioning & Alignment
```

```
d_inst_interp_period = d_inst_clock_period / d_interps_per_symbol;
```

```
// Tag Propagation
```

```
d_inst_output_period = d_inst_clock_period / d_osps;
```

```
}
```

PI 제어기

$$\text{avg_period}[n] = \text{avg_period}[n-1] + \beta \cdot e[n]$$

$$\text{inst_period}[n] = \text{avg_period}[n] + \alpha \cdot e[n]$$

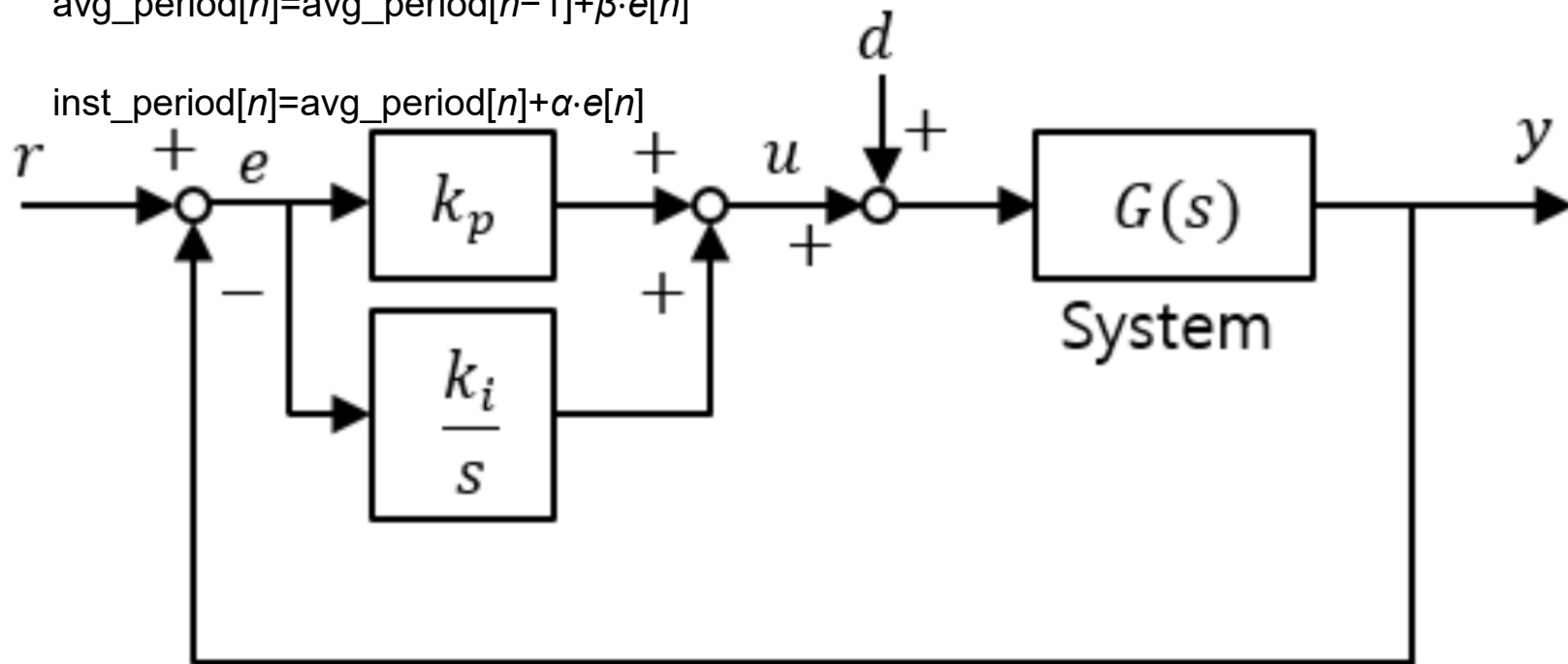


그림. 1 PI 제어기의 블록 선도

PI제어기

```
void advance_loop(float error)
```

```
{
```

```
    // So the loop can be reverted one step, if needed.
```

```
    d_prev_avg_period = d_avg_period;
```

```
    d_prev_inst_period = d_inst_period;
```

```
    d_prev_phase = d_phase;
```

```
// Integral arm of PI filter
```

```
d_avg_period = d_avg_period + d_beta * error;
```

```
// Limit the integral arm output here, as a large negative
```

```
// error input can lead to a negative d_avg_period, which
```

```
// will cause an infinite loop in the phase wrap method.
```

```
period_limit();
```

```
// Proportional arm of PI filter and final sum of PI filter arms
```

```
d_inst_period = d_avg_period + d_alpha * error;
```

```
// Limit the filter output here, for the errant case of a large
```

```
// negative error input, that can lead to a negative d_inst_period,
```

```
// which results in an incorrect phase increment, as it is assumed
```

```
// to be moving forward to the next symbol.
```

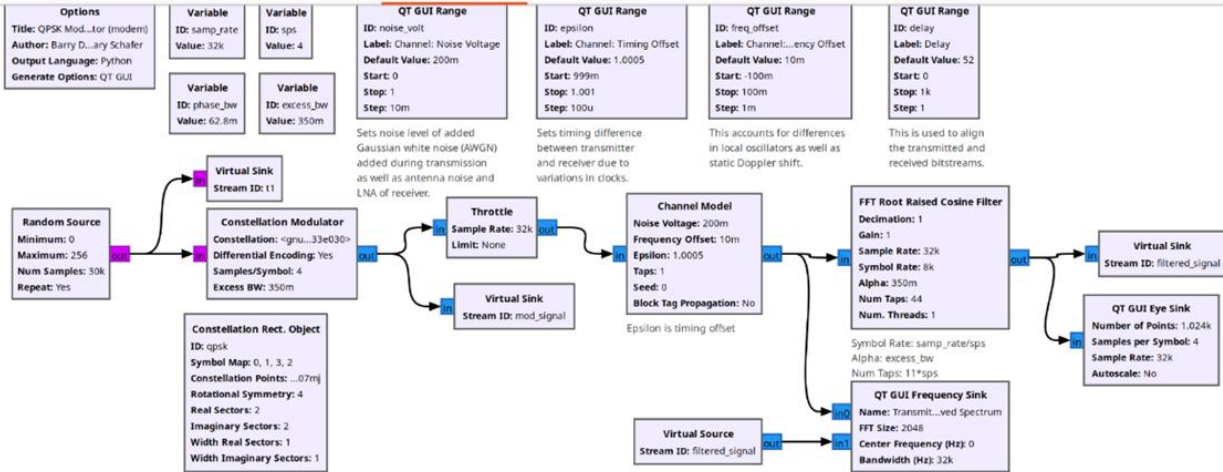
```
if (d_inst_period <= 0.f)
```

```
    d_inst_period = d_avg_period;
```

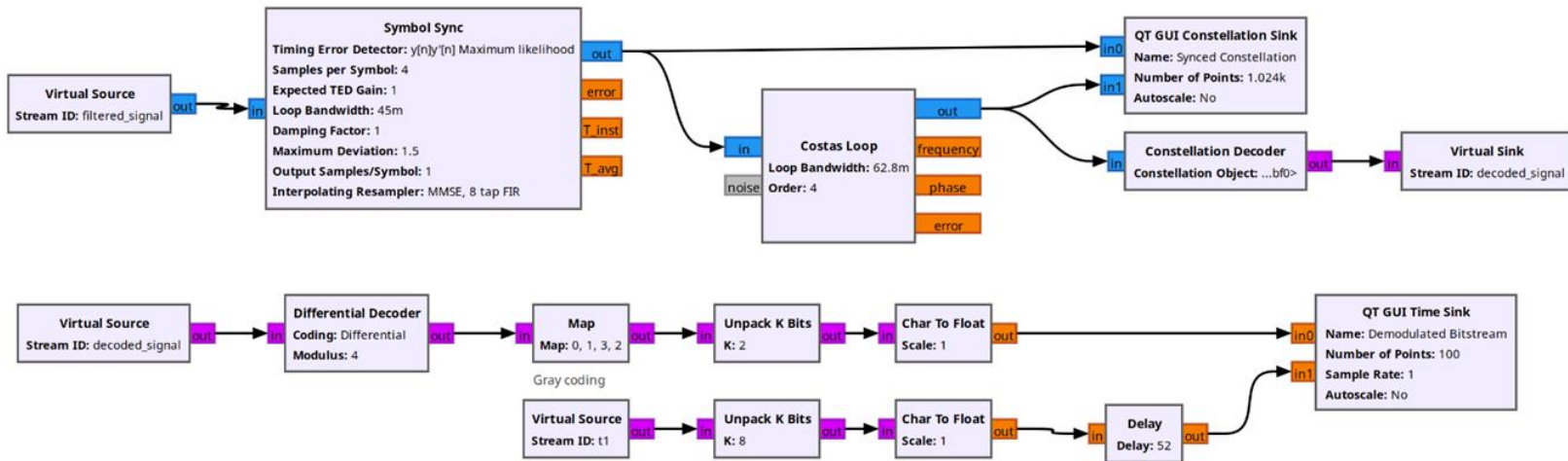
```
// Compute the new, unwrapped clock phase
```

```
d_phase = d_phase + d_inst_period;
```

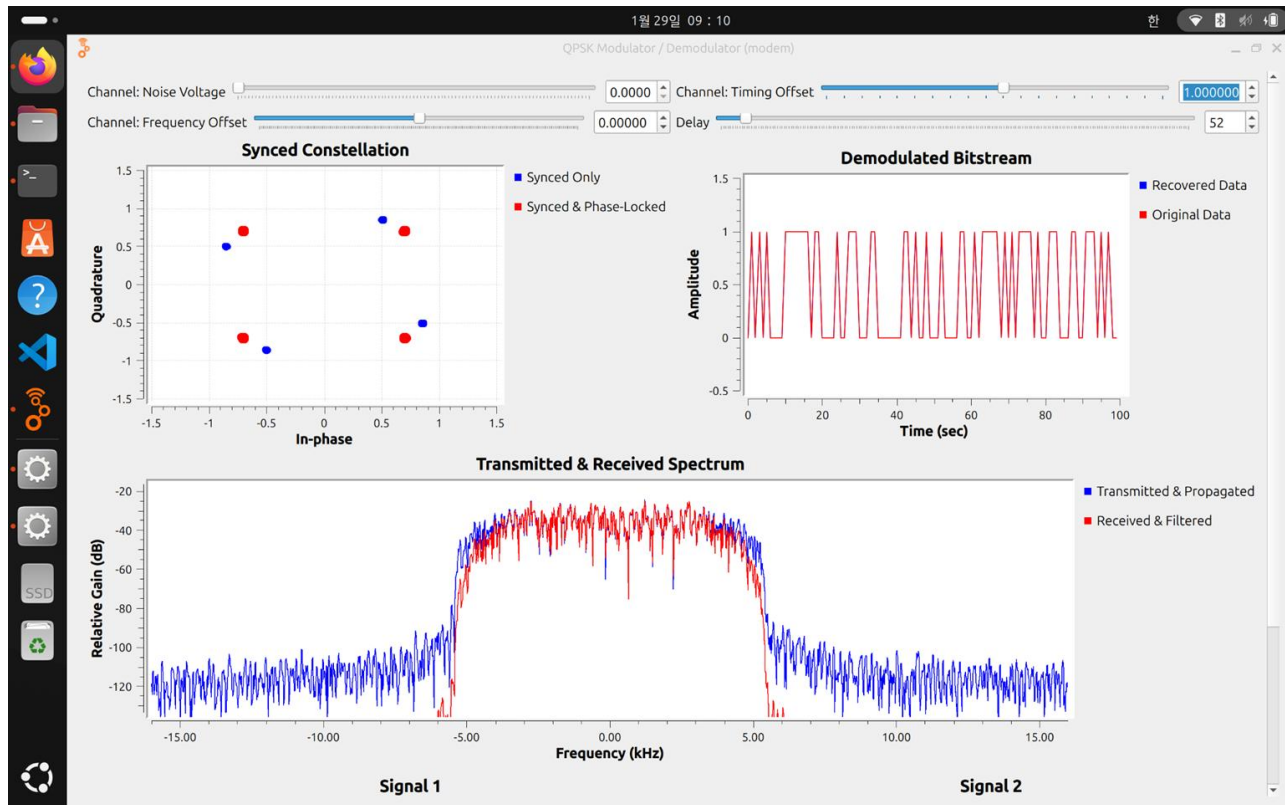
```
}
```



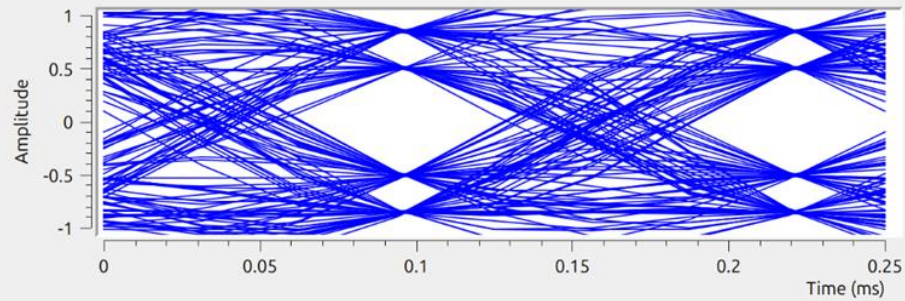
Symbol Map: [0,1,3,2] <- This is Gray coding
Constellation Points: [-0.707+0.707j, 0.707+0.707j, -0.707-0.707j, 0.707-0.707j]



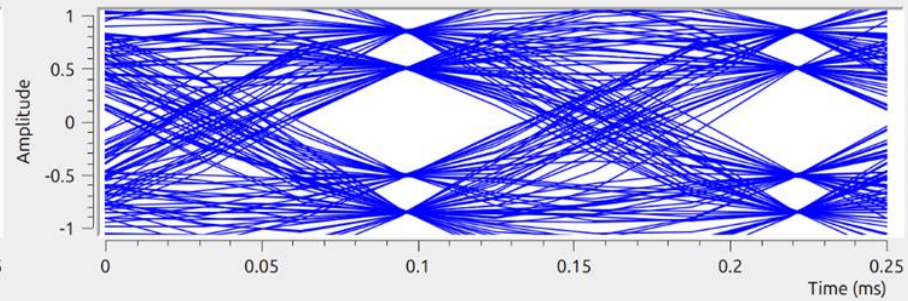
freq offset 0, timing offset 1



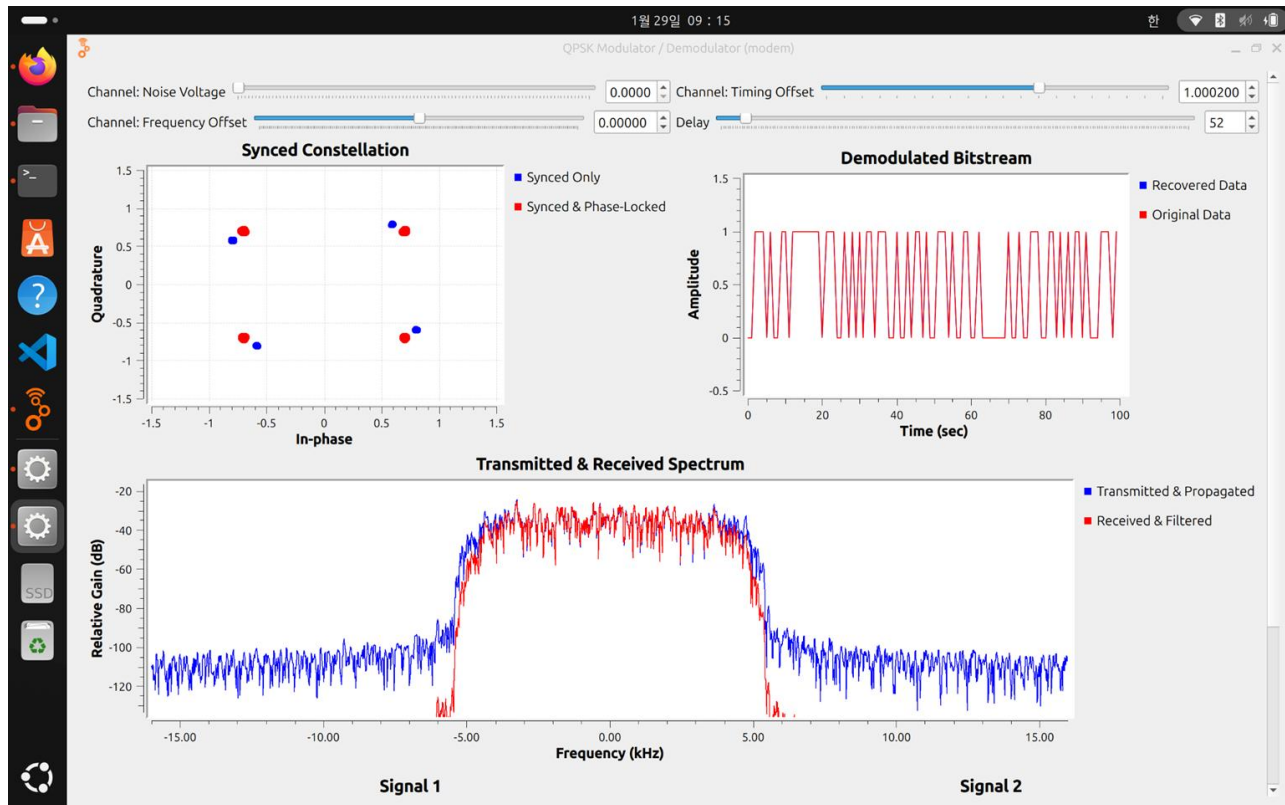
Signal 1

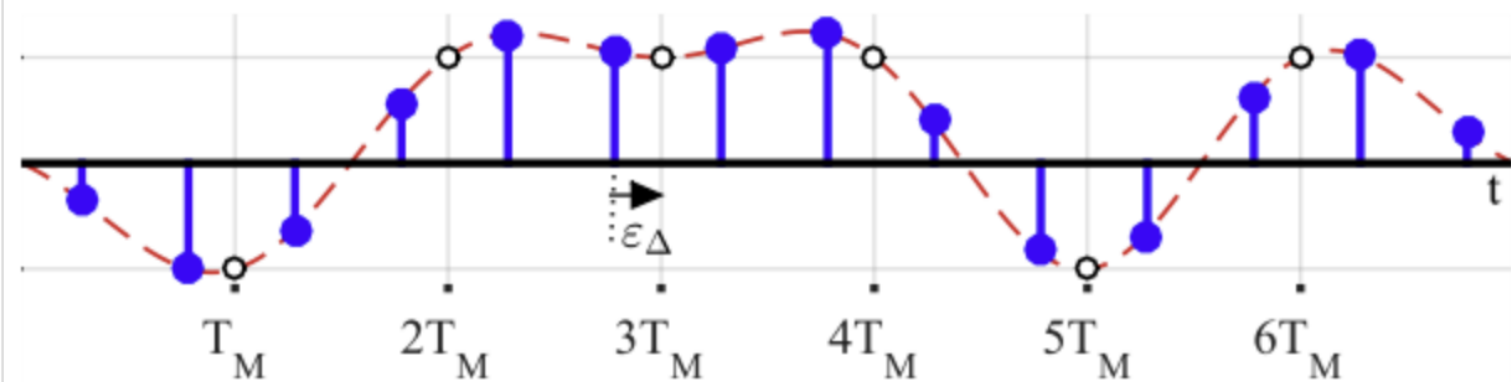
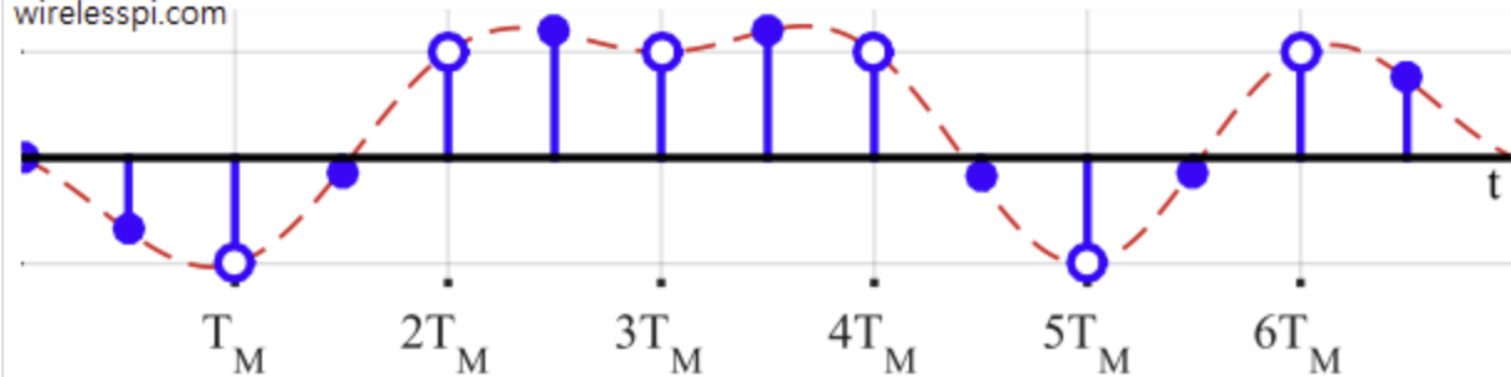


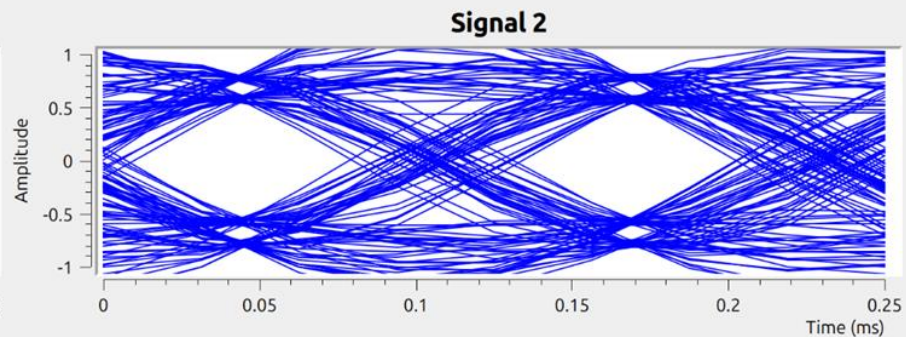
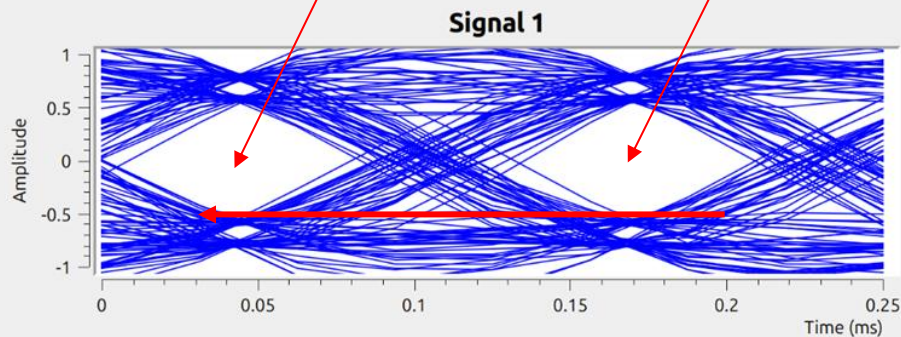
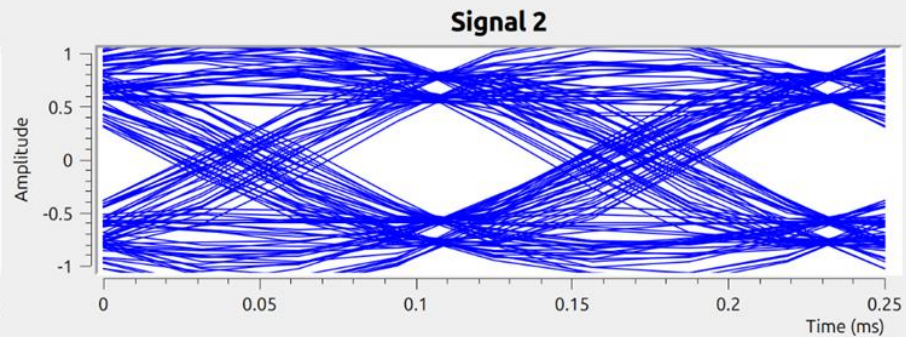
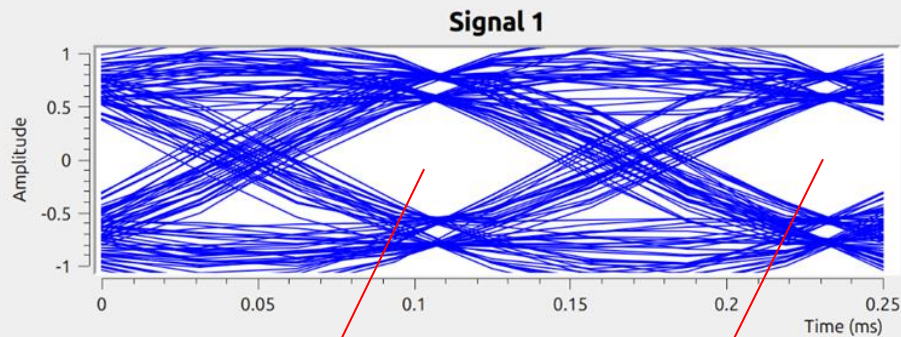
Signal 2



freq offset 0, timing offset 1.0002

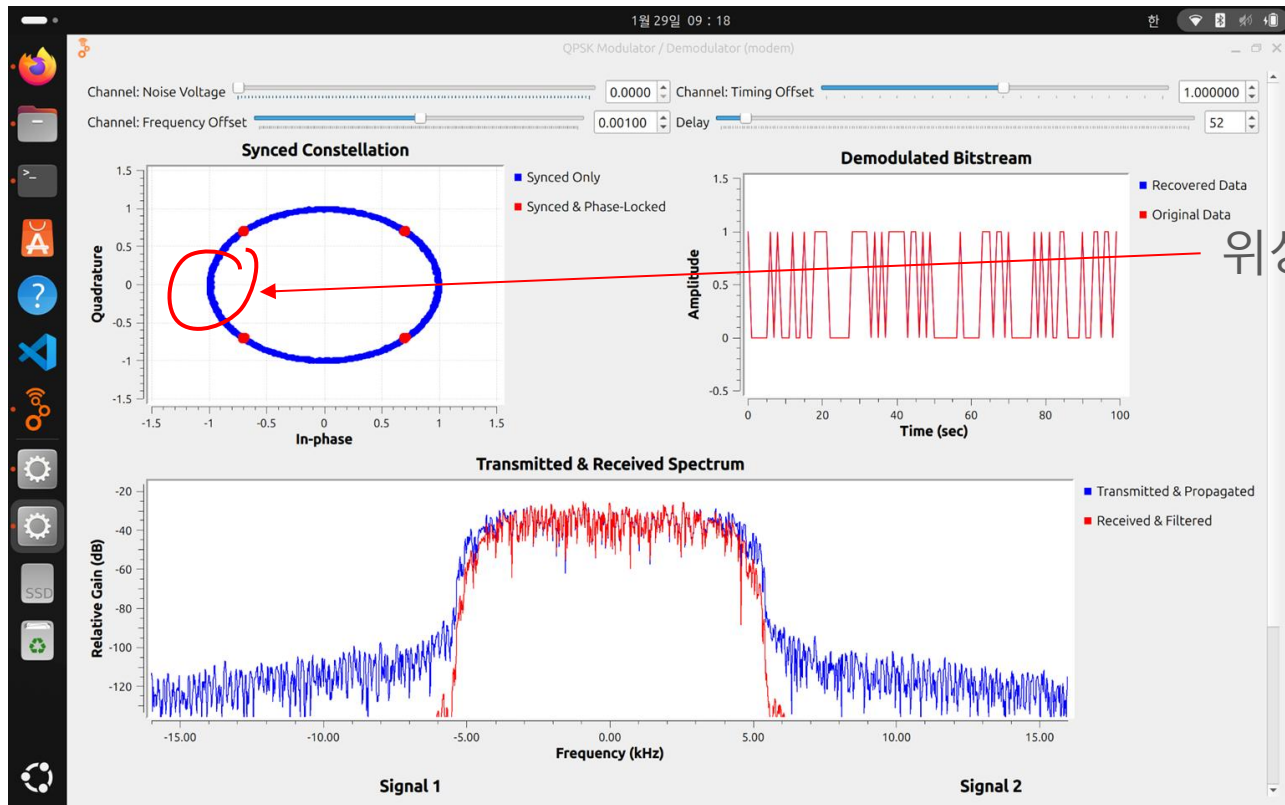




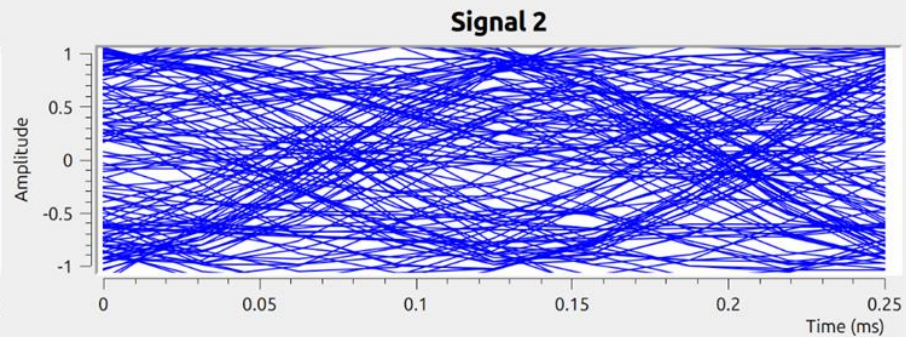
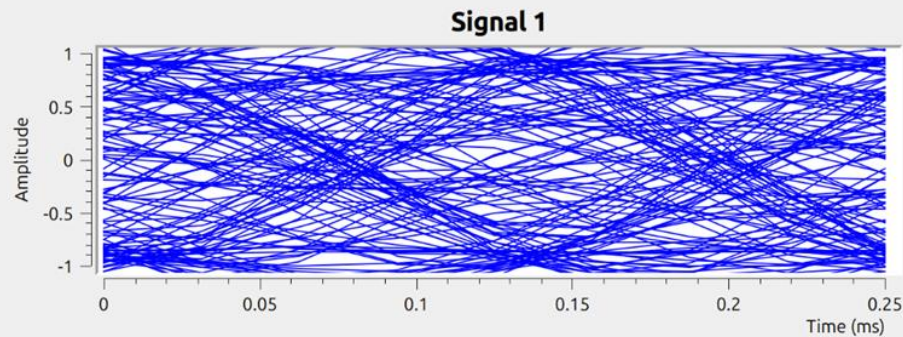


<symbol sync 보정 전, tx단에서의 eye diagram>

freq offset=0.001, timing offset=1



위상이 계속 변함을 확인 가능



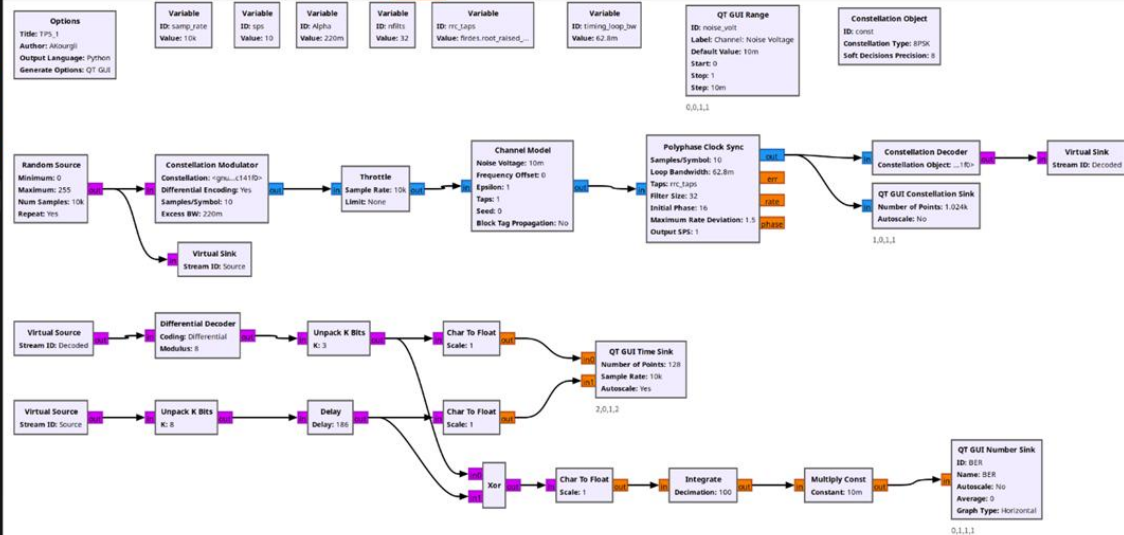
수신 신호의 위상이 시간에 따라 계속 변함에 따라 겹쳐진 여러 수신신호들이 뒤섞인 것 처럼 보인다.

8-psk

File Edit View Run Tools Help



bpsk2 x qpsk x Qpsk_modem2 x zmq_bpsk x test x Bpsk x



Core

- > Audio
- > Boolean Operators
- > Byte Operators
- > Channel Models
- > Channelizers
- > Coding
- > Control Port
- > Debug Tools
- > Deprecated
- > Digital Television
- > Equalizers
- > Error Coding
- > File Operators
- > Filters
- > Fourier Analysis
- > GUI Widgets
- > IQ Correction
- > Impairment Models
- > Industrial I/O
- > Instrumentation
- > Level Controllers
- > Math Operators
- > Measurement Tools
- > Message Tools
- > Misc
- > Modulators
- > Networking Tools
- > OFDM
- > Packet Operators
- > Peak Detectors
- > Resamplers
- > Soapy
- > Stream Operators
- > Stream Tag Tools
- > Symbol Coding
- > Synchronizers
- > Trellis Coding
- > Type Converters
- > UHD
- > Variables
- > Video
- > Waveform Generators
- > ZeroMQ Interfaces

Generating: "/home/jung/Downloads/TPS_1.py"

Executing: /usr/bin/python3 -u /home/jung/Downloads/TPS_1.py

QStandardPaths: runtime directory '/run/user/1000' is not owned by UID 0, but a directory permissions 0700 owned by UID 1000 GUID 1000

QSocketNotifier: Can only be used with threads started with QThread

QStandardPaths: runtime directory '/run/user/1000' is not owned by UID 0, but a directory permissions 0700 owned by UID 1000 GUID 1000

>>> Done

ID Value

Imports

Variables

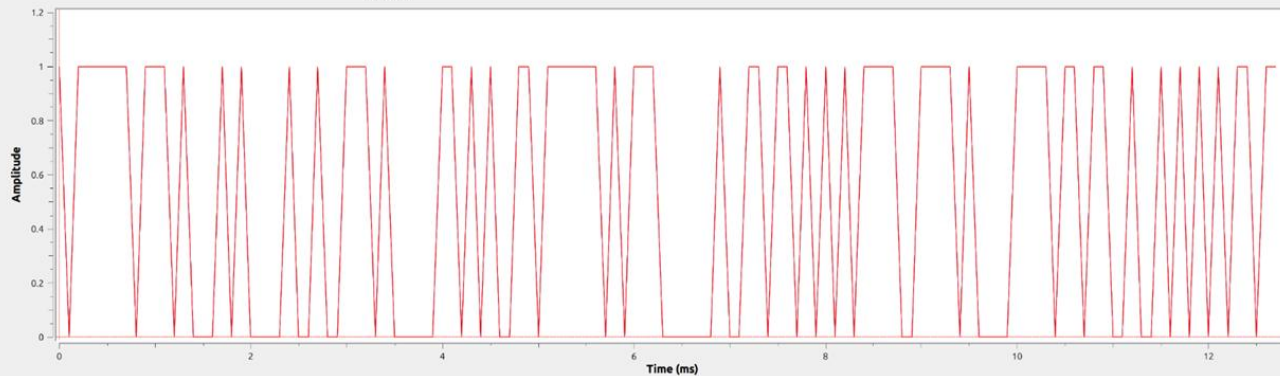
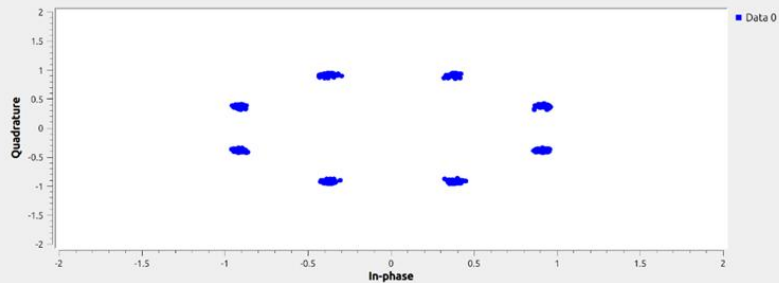
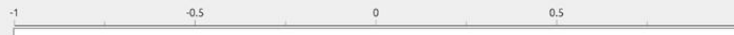
Alpha	0.22
const	<gnuradio.digital.digital_python.constellation_bpsk object at 0x76c1f4dbaf0>
nfft	32
noise_volt	0.01
rrc_taps	[0.0005638407892547548, 0.0005536819808185101, 0.0005434030899778008, 0.0005330053390935063, 0.0005224900669418275, 0.0005118585540913045, 0.0005011121975257993,
samp_rate	10000
sps	10

BER

Channel: Noise Voltage

0.01

Data 0 0.000000



■ Unmapped Decoded Bits Axes
■ Source Bits

☒ Autoscale☐ Grid☒ Axis Labels

Y Offset: + -

Y Range: + -

X Max: + -

Trigger

Auto

Positive

Level: + -

Delay: + -

Extras

Autoscale

Stop